

实验 3 地图构建

一、实验目的

- 1、掌握 gazebo 仿真平台的使用方法，了解机器人模型的碰撞参数、惯性矩阵设置、运动控制、传感器配置和仿真环境搭建。
- 2、熟悉 ROS 中 gmapping、hector_slam 建图功能包的工作原理，掌握其节点结构、参数配置、启动文件编写与仿真数据联调方法。
- 3、理解基于激光雷达的栅格地图构建原理，包括占据率更新、扫描匹配、坐标变换与地图更新逻辑，了解 SLAM 建图流程。
- 4、熟练使用 map_server 功能包，掌握栅格地图的保存、加载与参数查看方法。

二、实验环境与器材

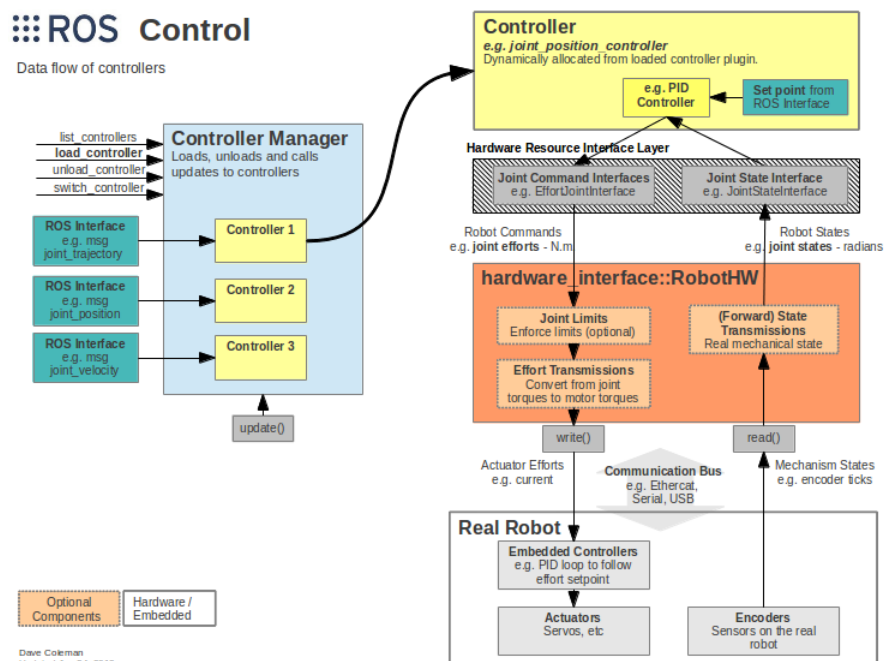
- 1、实验硬件设备
计算机、移动机器人导航平台
- 2、实验软件配置
Ubuntu20.04, ros-noetic
- 3、实验场景
教学实验场地

三、实验原理

1. ros_control

ros_control 是 ROS 中用于机器人硬件控制的核心框架，由一组软件包组成，包含控制器接口、控制器管理器、传输和 hardware_interfaces 组件，提供了一套标准化的控制接口，简化开发者从仿真到实体机器人的开发流程。

Gazebo 仿真环境已经实现了 ros_control 相关的硬件接口，如果需要在 Gazebo 中控制机器人运动，调用相关插件即可充当硬件接口，模拟机器人物理行为。完成开发后，只需将硬件接口替换为真实机器人的驱动，即可直接部署到实体机器人上，极大地提高了开发效率和代码的可移植性。

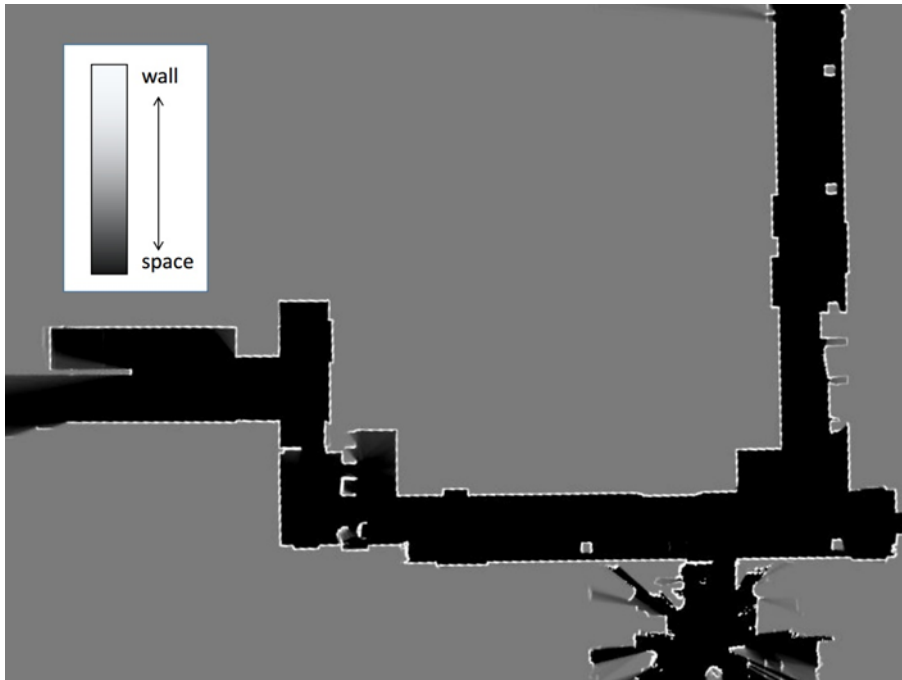


官方网址: https://wiki.ros.org/ros_control

2. 全局地图构建

地图是机器人导航系统中一个重要的组成元素,它可以解决机器人在未知环境中“我在哪”和“周围环境是什么”的两个基本问题,并为后续的路径规划算法提供搜索空间。即机器人在运动过程中,需要参考一张全局性的地图,来确定自身的位置、目的地位置,也需要根据地图来规划出一条去往目的地的大致路线。

要使用地图,首先必须要绘制地图,地图绘制或构建并非静态的扫描,而是一个动态的、概率估计的过程,其核心技术是即时定位与地图构建,简称 SLAM (simultaneous localization and mapping)。SLAM 技术让机器人能够在未知环境中一边移动,一边绘制出周围环境的地图,并同时确定自己在这张地图中的精确位置。在 ROS 中,常用的 SLAM 实现算法也比较多,比如:gmapping、hector_slam、cartographer、ORB_SLAM、rgbdslam 等。此外,通过 SLAM 完成地图构建时,机器人必须具备感知外界环境的能力,特别是获取周围环境深度信息的能力,感知的实现需要依赖于传感器,比如:激光雷达、里程计/IMU、视觉传感器、RGB-D 摄像头等。本实验中小车配置 2D 激光雷达用于构建 2D 栅格地图(类似建筑平面图,如下图所示),选用的 SLAM 算法为 gmapping 和 hector_mapping。



1) gmapping

gmapping 是 ROS 开源社区中较为常用且比较成熟的 2D 激光 SLAM 算法之一,核心思想是利用粒子滤波来估计机器人的运动轨迹,同时基于估计出的轨迹构建栅格地图,算法在执行过程中需要依赖移动机器人的里程计数据和激光雷达数据,因此,要求该移动机器人可以发布里程计消息和雷达消息。

gmapping 功能包的安装命令: `sudo apt install ros-<ROS 版本>-gmapping`。

gmapping 功能包的核心节点是 `slam_gmapping`,该节点订阅的话题、发布的话题、服务以及相关参数可查阅网址: <https://wiki.ros.org/gmapping>。

gmapping 功能包的使用需编写与 gmapping 节点相关 launch 文件,可参考 github 的演示 launch 文件: https://github.com/ros-perception/slam_gmapping/blob/melodic-devel/gmapping/launch/slam_gmapping_pr2.launch

2) hector_slam

Hector SLAM 也是一种经典的 2D 激光 SLAM 算法，由德国达姆施塔特工业大学的 Team Hector 开发，核心特点是**完全不依赖里程计信息**，仅通过激光雷达的扫描匹配来实现定位与建图。算法的核心原理是：基于优化的扫描匹配，即寻找一个最优的位姿变换，使得当前的激光扫描数据能与已有的地图达到最佳对齐。

hector_slam 功能包的安装命令：sudo apt install ros-<ROS 版本>-hector-slam。

hector_slam 功能包的核心节点是 hector_mapping，该节点订阅的话题、发布的话题、服务以及相关参数可查阅网址：https://wiki.ros.org/hector_mapping。

hector_slam 功能包的使用需编写与 hector_mapping 节点相关 launch 文件，可参考 github 的演示 launch 文件：https://github.com/tu-darmstadt-ros-pkg/hector_slam/blob/noetic-devel/hector_mapping/launch/mapping_default.launch

3. 栅格地图

栅格地图也叫占据栅格地图 (Occupancy Grid Map)，在通常的尺度地图中，对于一个点，它要么有 (Occupied 状态，下面用 1 来表示) 障碍物，要么没有 (Free 状态，下面用 0 来表示) 障碍物。在占据栅格地图中，对于一个点，我们用 $p(s=1)$ 来表示它是 Occupied 状态的概率，用 $p(s=0)$ 来表示它是 Free 状态的概率，当然两者的和为 1。用两个值表示一个点的状态有点冗余，我们引入两者的比值来作为点的状态：

$$odd(s) = \frac{p(s=1)}{p(s=0)}$$

对于一个点，得到新的测量值 (Measurement, $z \sim \{0,1\}$) 后，我们需要更新它的状态。假设测量值来之前，该点的状态为，我们要更新它为：

$$odd(s|z) = \frac{p(s=1|z)}{p(s=0|z)}。$$

根据贝叶斯公式，我们有：

$$p(s=1|z) = \frac{p(z|s=1)p(s=1)}{p(z)}$$

$$p(s=0|z) = \frac{p(z|s=0)p(s=0)}{p(z)}$$

带入之后得：

$$\begin{aligned} odd(s|z) &= \frac{p(s=1|z)}{p(s=0|z)} \\ &= \frac{p(z|s=1)p(s=1)/p(z)}{p(z|s=0)p(s=0)/p(z)} \\ &= \frac{p(z|s=1)}{p(z|s=0)} odd(s) \end{aligned}$$

对两边取对数得：

$$\log odd(s|z) = \log \frac{p(z|s=1)}{p(z|s=0)} + \log odd(s)$$

这样，含有测量值的项就只剩下 $\log \frac{p(z|s=1)}{p(z|s=0)}$ 了，我们称这个比值为测量值的模型

(Measurement Model)，标记为 $lomeas$ ，测量值的模型只有两种： $lofree = \log \frac{p(z=0|s=1)}{p(z=0|s=0)}$ 和

$looccu = \log \frac{p(z=1|s=1)}{p(z=1|s=0)}$ ，且均为定值。

这样，如果我们用 $\log Odd(s)$ 来表示栅格 s 的状态 S 的话，我们的更新规则就进一步

简化成了： $S^+ = S^- + \log \frac{p(z|s=1)}{p(z|s=0)}$ ，其中 S^+ 和 S^- 分别表示测量值之后和之前栅格 s 的

状态。

此外，一个栅格的初始状态 S_{init} 为：默认栅格空闲和栅格占据的概率都为 0.5。

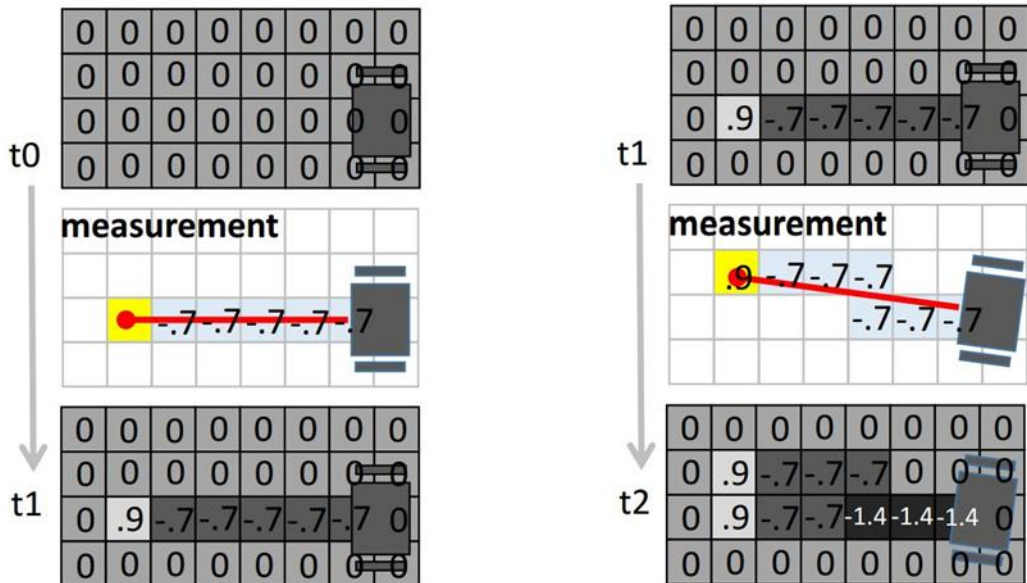
$$S_{init} = Odd(s) = \log \frac{p(s=1)}{p(s=0)} = \log \frac{0.5}{0.5} = 0$$

此时，经过这样的建模，更新一个栅格的状态只需要做简单的加法即可。

$$S^+ = S^- + lofree \text{ 或 } S^+ = S^- + looccu$$

举例验证占据栅格地图构建算法：

首先，我们假设 $looccu = 0.9$ ， $lofree = -0.7$ 。那么，显而易见，一个栅格状态的数值越大，就越表示该栅格为占据状态，相反数值越小，就表示该栅格为空闲状态。（这也就解决了此前文中提出的激光雷达观测值“不一定准”的问题）



上图是用两次激光扫描数据更新地图的过程。在结果中，颜色越深越表示栅格是空闲的，颜色越浅越表示是占据的。这里要区分常用的激光 SLAM 算法中的地图，只是表述方式的不同，没有对错之分。

4. 地图服务

SLAM 算法（如 gmapping 或 hector_slam）构建的地图数据保存在内存中，虽然可以在 RViz 中显示，但当节点关闭时，数据会被一并释放，下次如需使用地图需重新构建。针对这一问题，ROS 中提供了地图服务功能包 map_server，方便栅格地图的保存和读取。

map_server 功能包的安装命令：sudo apt install ros-<ROS 版本>-map-server。

map_server 功能包提供了两个节点：map_saver 和 map_server，map_saver 用于将栅格地图保存至磁盘，map_server 读取磁盘上的静态栅格地图加载到内存，并以服务的方式提供给导航栈或其他节点使用。相关节点订阅的话题、发布的话题、服务以及相关参数可查阅网址：

https://wiki.ros.org/map_server。

map_server 功能包的使用可以运行 rosrun 命令或编写与节点相关 launch 文件。

四、实验内容

1、仿真实验

内容描述：加载带有激光雷达的机器人小车模型和 gazebo 仿真环境，键盘控制仿真小车运动，完成仿真环境的地图构建并进行保存。

具体流程如下：

第 1 步：安装 gmapping 和 map_server 功能包

gmapping 包：用于构建导航实验场景的二维栅格地图。

map_server 包：将 gmapping 构建的地图数据保存至磁盘。

在终端上运行以下命令：

```
$ sudo apt install ros-noetic-gmapping
$ sudo apt install ros-noetic-map-server
```

第 2 步：gazebo 中载入机器人模型和搭建的仿真环境

1) 创建 ros 功能包 robot_sim

进入工作空间的 src 文件夹，创建功能包 robot_sim，并添加依赖 urdf、xacro、gazebo_ros 和 gazebo_plugins，在终端上运行以下命令：

```
$ cd ~/catkin_robot_sim/src
$ catkin_create_pkg robot_sim urdf xacro gazebo_ros gazebo_plugins
```

2) 添加机器人模型和仿真环境文件

将 demo03_maping/sim 文件夹里面的 urdf、worlds 两个文件夹直接复制到 robot_sim 文件夹下，文件夹里是 urdf 文件构建的机器人模型和 gazebo 构建的仿真环境。

```
$ cp -r /path/to/sim/urdf ~/catkin_ebot_sim/src/robot_sim
$ cp -r /path/to/sim/worlds ~/catkin_ebot_sim/src/robot_sim
```

3) 编写 launch 文件

编写启动 gazebo 并载入机器人模型和仿真环境的 launch 文件 robot_sim_gazebo.launch，在终端上运行以下命令：

```
$ cd ~/catkin_ebot_sim/src/robot_sim
$ mkdir launch && cd launch
$ gedit robot_sim_gazebo.launch
```

robot_sim_gazebo.launch 文件输入内容示例以下：

```
<launch>
  <!-- 0. 定义参数 -->
  <!-- 设置仿真时间 -->
  <arg name="use_sim_time" default="true"/>

  <!-- 1.在参数服务器载入 urdf 文件 -->
  <param name="robot_description" command="$(find xacro)/xacro $(find
robot_sim)/urdf/xacro00 car all.urdf.xacro" />

  <!-- 2.启动 gazebo, 并加载构建的仿真环境 -->
  <include file="$(find gazebo_ros)/launch/empty_world.launch" >
    <arg name="world_name" value="$(find robot_sim)/worlds/
box_house.world" />
    <arg name="use_sim_time" value="$(arg use_sim_time)"/>
  </include>

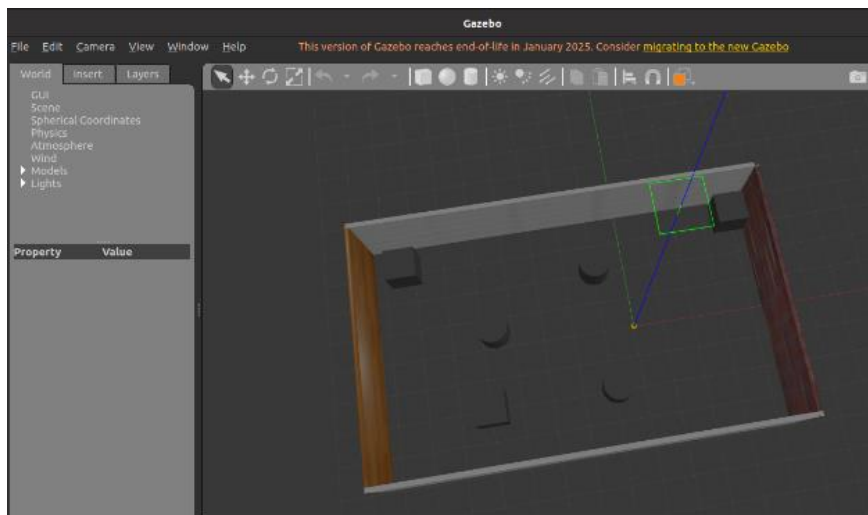
  <!-- 3.在 gazebo 中加载一个机器人模型-->
  <node pkg="gazebo_ros" type="spawn_model" name="spawn_model"
args="-urdf -model mycar -param robot_description" output="screen" />

</launch>
```

5) 编译执行

在终端上运行以下命令：

```
$ cd ~/catkin_ebot_sim
$ catkin_make
$ source ./devel/setup.bash
$ roslaunch robot_sim robot_sim_gazebo.launch
```



启动键盘控制节点遥制机器人运动，gazebo 中的机器人模型按照操作运动，说明机器人和仿真环境加载成功。

第 3 步：配置 gmapping 建图环境

1) 创建 ros 功能包 ebot_bringup

进入 src 文件夹, 创建 ros 功能包 ebot_bringup, 并添加依赖 gmapping 和 map_server,

在终端上运行以下命令：

```
$ cd ~/catkin_ebot_sim/src
$ catkin_create_pkg ebot_bringup gmapping map_server
```

2) 编写 launch 文件

编写 bringup.launch 以及与 gmapping 节点相关的 launch 文件。bringup.launch 用于加载机器人模型和 gazebo 仿真环境，并启动 rviz。

在终端上运行以下命令：

```
$ cd ~/catkin_ebot_sim/src/ebot_bringup
$ mkdir launch && cd launch
$ gedit bringup.launch
$ gedit slam_gmapping.launch
```

bringup.launch 文件输入内容示例如下：

```
<launch>
  <!-- 0. 定义参数 -->
  <!-- 仿真环境下，将该参数设置为 true -->
  <arg name="use_sim_time" default="true"/>
  <param name="use_sim_time" value="$(arg use_sim_time)"/>

  <!-- 1. 载入机器人模型和仿真环境 -->
  <include file="$(find robot_sim)/launch/robot_sim_gazebo.launch" >
    <arg name="use_sim_time" value="$(arg use_sim_time)"/>
  </include> -->

  <!-- 2. 启动机器人状态和关节状态发布节点 -->
  <node pkg="joint_state_publisher" name="joint_state_publisher"
type="joint_state_publisher" />
  <node pkg="robot_state_publisher" name="robot_state_publisher"
type="robot_state_publisher" />

  <!-- 3. 启动 rviz -->
  <node pkg="rviz" type="rviz" name="rviz" />
</launch>
```

slam_gmapping.launch 文件输入内容示例如下：

```
<launch>
  <!-- gmapping 节点 -->
  <node pkg="gmapping" type="slam_gmapping" name="slam_gmapping"
output="screen">
    <remap from="scan" to="scan"/>
    <param name="base_frame" value="base_footprint"/><!--底盘坐标系-->
    <param name="odom_frame" value="odom"/> <!--里程计坐标系-->
    <param name="map_frame" value="map"/> <!--地图坐标系-->
    <param name="map_update_interval" value="5.0"/>
    <param name="maxUrange" value="16.0"/>
    <param name="sigma" value="0.05"/>
    <param name="kernelSize" value="1"/>
    <param name="lstep" value="0.05"/>
    <param name="astep" value="0.05"/>
    <param name="iterations" value="5"/>
    <param name="lsigma" value="0.075"/>
    <param name="ogain" value="3.0"/>
    <param name="lskip" value="0"/>
  </node>
</launch>
```



```

<param name="srr" value="0.1"/>
<param name="srt" value="0.2"/>
<param name="str" value="0.1"/>
<param name="stt" value="0.2"/>
<param name="linearUpdate" value="1.0"/>
<param name="angularUpdate" value="0.5"/>
<param name="temporalUpdate" value="3.0"/>
<param name="resampleThreshold" value="0.5"/>
<param name="particles" value="30"/>
<param name="xmin" value="-50.0"/>
<param name="ymin" value="-50.0"/>
<param name="xmax" value="50.0"/>
<param name="ymax" value="50.0"/>
<param name="delta" value="0.05"/>
<param name="lssamplerange" value="0.01"/>
<param name="lssamplestep" value="0.01"/>
<param name="lasamplerange" value="0.005"/>
<param name="lasamplestep" value="0.005"/>
</node>
</launch>

```

注: slam_gmapping.launch 配置内容, 可参阅 gmapping 源码的 launch 文件。gmapping 源码的下载地址: https://github.com/ros-perception/slam_gmapping。

第 4 步: 构建地图并保存

1) 编译工作空间

编译并加载环境变量, 在终端上运行以下命令:

```

$ cd ~/catkin_ebot_sim
$ catkin_make
$ source ./devel/setup.bash

```

2) 键控机器人构建地图

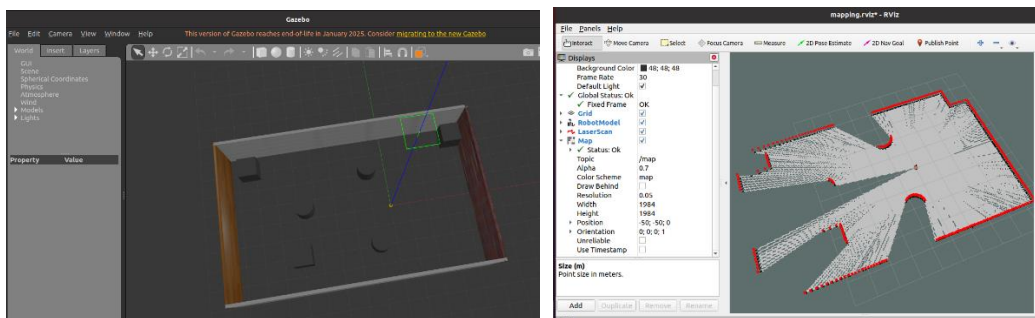
加载机器人模型并启动 rviz 和 gazebo 仿真环境, 终端 1 上运行以下命令:

```
$ roslaunch ebot_bringup bringup.launch
```

启动 gmapping 绘图节点, 终端 2 上运行以下命令:

```
$ roslaunch ebot_bringup slam_gmapping.launch
```

在启动的 rviz 中, 添加 RobotModel、TF 显示, 可视化机器人模型与坐标变换关系; 添加 LaserScan 显示, 设置 topic 为 /scan, 可视化雷达激光点; 添加 Map 显示, 设置 topic 为 /map, 可视化栅格地图。



启动键盘遥控节点, 用于控制机器人运动建图, 终端 3 上运行以下命令:

启动键盘遥控节点，终端 3 上运行以下命令：

```
$ rosrn teleop_twist_keyboard teleop_twist_keyboard.py
```

通过键盘控制机器人模型运动，完成仿真环境的地图构建。

注：同时启动 RViz 和 Gazebo 对硬件资源消耗过大，电脑会出现卡顿现象，通过 wsl 安装的 ubuntu 系统更为严重，会出现 RViz 和 Gazebo 的操作界面延迟过高导致整个系统卡死的情况，建议修改 robot_sim_gazebo.launch 文件，不启动 Gazebo 的 GUI 界面，修改内容示例如下：

```
<arg name="gui" default="false"/>

<!-- 2.启动 gazebo，并加载构建的仿真环境 -->
<include file="$(find gazebo_ros)/launch/empty_world.launch" >
  <arg name="world_name" value="$(find robot_sim)/worlds/
box_house.world" />
  <arg name="use_sim_time" value="$(arg use_sim_time)"/>
  <arg name="gui" value="$(arg gui)"/>
  <arg name="paused" value="false"/>
  <arg name="debug" value="false"/>
</include>
```

3) 保存地图

地图构建完成后使用 map_server 包提供的 map_saver 工具将当前内存中的地图保存到磁盘。终端 4 输入如下命令：

```
$ cd ~/catkin_ebot_sim/src/ebot_bringup
$ mkdir maps && cd maps
$ rosrn map_server map_saver -f nav
```

运行结果：在指定的 maps 文件夹下会生成两个文件，nav.pgm 与 nav.yaml。

2、实车实验

内容描述：启动机器人小车底盘，启动激光雷达传感器，键盘控制小车运动，完成教学实验场地的地图构建并进行保存。

具体流程如下：

1) 查看雷达硬件连接

在宿主机命令端运行以下命令：

```
$ ls -l /dev/serial/by-path/
```

启动容器，在容器终端运行以下命令：

```
$ ls -l /dev/serial/by-path/
```

如果两个终端有如下输出，说明容器和底盘、雷达连接正常。

```
nvidia@nvidia-desktop:~$ ls -l /dev/serial/by-path/
total 0
lrwxrwxrwx 1 root root 13 3月 17 18:31 platform-3610000.usb-usb-0:2.3:1.0-port0
-> ../../ttyUSB0
lrwxrwxrwx 1 root root 13 3月 17 18:31 platform-3610000.usb-usb-0:2.4:1.0 -> ..
/../../ttyACM0
```

```
nvidia@nvidia-desktop:~$ docker exec -it noetic_demo bash
root@nvidia-desktop:/home/nvidia# ls -l /dev/serial/by-path/
total 0
lrwxrwxrwx 1 root root 13 Mar 17 10:31 platform-3610000.usb-usb-0:2.3:1.0-port0
-> ../../ttyUSB0
lrwxrwxrwx 1 root root 13 Mar 17 10:31 platform-3610000.usb-usb-0:2.4:1.0 -> ../../
../../ttyACM0
```

2) 安装 hector_mapping 和 map_server 功能包

查看容器中是否已安装了 hector_mapping 和 map_server，容器终端运行以下命令：

```
$ rospack find hector_mapping
$ rospack find map_server
```

如果输出功能包的安装路径，说明已经安装。如果没有，容器终端运行以下命令进行安装：

```
$ sudo apt install ros-noetic-hector-mapping
$ sudo apt install ros-noetic-map-server
```

3) 配置雷达驱动

将 demo03_mapping/ebot 文件夹里面的 ydlidar_ros_driver 文件夹直接复制到工作空间的 src 文件夹下，ydlidar_ros_driver 为雷达的驱动功能包，部署了雷达传感器驱动节点，并配置了相应的 TF 坐标变换和消息接口，使用过程中可以通过修改功能包里 launch 文件夹下的 lidar.launch 文件，定义 ROS 与激光雷达的交互与通信方式。

在实际应用中，机器人系统通常采用多传感器配置，除主雷达外，还会集成其他类型的雷达或传感器，为了方便功能包的管理，可以在 src 文件夹下新建文件夹 ebot_sensors，然后将 ydlidar_ros_driver 功能包移入。

在终端上运行以下命令完成上述操作步骤：

```
$ cd ~/test_demo/catkin_ebot/src
$ mkdir ebot_sensors && cd ebot_sensors
$ cp -r /path/to/ebot/ydlidar_ros_driver .
$ cd ../ebot_bringup/launch/
$ cp ../../ebot_sensors/ydlidar_ros_driver/launch/lidar.launch my_lidar.launch
$ gedit my_lidar.launch
```

根据雷达的规格书修改 my_lidar.launch 文件中部分参数，此外，也需发布雷达和小车底盘的坐标变换，示例如下：

```
<param name="frame_id" type="string" value="laser_link"/>
<!-- 雷达连接的串口设备路径 -->
<param name="port" type="string" value="/dev/serial/by-path/platform-3610000.usb-usb-0:2.3:1.0-port0"/>
<!-- 串口通信的波特率，改为“115200” -->
<param name="baudrate" type="int" value="115200"/>
<!-- 采样频率，改为“3”，即每秒采集 3000 点 -->
<param name="sample_rate" type="int" value="3"/>
<!-- 是否镜像翻转雷达的扫描数据 -->
```

```

<param name="reversion" type="bool" value="false"/>
<!-- 是否为单通道雷达 -->
<param name="isSingleChannel" type="bool" value="true"/>
<!-- 是否支持通过串口线的 DTR 引脚信号来控制电机转动 -->
<param name="support_motor_dtr" type="bool" value="true"/>
<!-- 最大测距范围 -->
<param name="range_max" type="double" value="8.0" />
<!-- 发布雷达与小车底盘的坐标变换 -->
<node pkg="tf2_ros" type="static_transform_publisher" name="base_link_to_laser" args="0.0 0.0 0.15 3.1415 0.0 0.0 /base_link /laser_link " />

```

4) 配置 hector_mapping 建图环境

在终端上运行以下命令：

```

$ cd ~/test_demo/catkin_ebot/src/ebot_bringup/launch
$ gedit slam_hecktor.launch

```

slam_hecktor.launch 文件输入内容示例如下：

```

<launch>
  <node pkg = "hector_mapping" type="hector_mapping"
name="hector_mapping" output="screen">
    <!-- Frame names -->
    <param name="map_frame" value="map" />
    <param name="base_frame" value="base_footprint" />
    <param name="odom_frame" value="base_footprint" />

    <!-- Tf use -->
    <param name="use_tf_scan_transformation" value="true"/>
    <param name="use_tf_pose_start_estimate" value="false"/>
    <param name="pub_map_odom_transform" value="true"/>

    <!-- Map size / start point -->
    <param name="map_resolution" value="0.05"/>
    <param name="map_size" value="2048"/>
    <param name="map_start_x" value="0.5"/>
    <param name="map_start_y" value="0.5" />
    <param name="map_multi_res_levels" value="2" />

    <!-- Map update parameters -->
    <param name="update_factor_free" value="0.4"/>
    <param name="update_factor_occupied" value="0.7" />
    <param name="map_update_distance_thresh" value="0.2"/>
    <param name="map_update_angle_thresh" value="0.06" />
    <param name="laser_z_min_value" value = "-1.0" />
    <param name="laser_z_max_value" value = "1.0" />

```

```

<!-- Advertising config -->
<param name="advertise_map_service" value="true"/>
<param name="scan_subscriber_queue_size" value="5"/>
<param name="scan_topic" value="scan"/>

<!-- Configure additional parameters -->
<param name="map_pub_period" value="2" />
<param name="laser_min_dist" value="0.4" />
<param name="laser_max_dist" value="5.5" />
<param name="output_timing" value="false" />
<param name="pub_map_scanmatch_transform" value="true" />
<param name="tf_map_scanmatch_transform_frame_name"
value="scanmatcher_frame" />
</node>

</launch>

```

注：slam_hector.launch 配置内容，可参阅 hector_mapping 源码的 launch 文件。

hector_mapping 源码的下载地址：https://github.com/tu-darmstadt-ros-pkg/hector_slam。

5) 编译并执行

编译并加载环境变量，在容器终端上运行以下命令：

```

$ cd ~/test_demo/catkin_ebot
$ catkin_make
$ source ./devel/setup.bash

```

启动小车底盘，在容器的第 1 个终端上运行以下命令：

```
$ roslaunch ebot_bringup bringup.launch
```

启动激光雷达，在容器的第 2 个终端上运行以下命令：

```
$ roslaunch ebot_bringup my_lidar.launch
```

启动 hector_mapping 绘图节点，在容器的第 3 个终端上运行以下命令：

```
$ roslaunch ebot_bringup slam_hector.launch
```

6) 键控机器人构建地图

启动 rviz，在远程 PC 机的第 1 个终端上运行以下命令：

```
$ rviz
```

在启动的 rviz 中，添加 TF、LaserScan 显示，可视化坐标变换关系与雷达扫描点；

添加 Map 显示，设置 topic 为/map，可视化栅格地图。

启动键盘遥控节点，在远程 PC 机的第 2 个终端上运行以下命令：

```
$ rosrn teleop_twist_keyboard teleop_twist_keyboard.py
```

通过键盘遥控机器人运动，完成教学实验场地的地图构建。

7) 保存地图

地图构建完成后使用 map_server 包提供的 map_saver 工具将当前内存中的地图保

存到磁盘。具体操作是在容器的第 3 个终端上运行以下命令:

```
$ cd ~/test_demo/catkin_ebot/src/ebot_bringup
$ mkdir maps && cd maps
$ rosrun map_server map_saver -f nav
```

运行结果: 在指定的 maps 文件夹下会生成两个文件, nav.pgm 与 nav.yaml。

注: 为了方便静态坐标变换的集中管理和维护, 建议将系统中所有静态坐标变换的发布指令写到同一个 launch 文件中, 且对于实体机器人, URDF 文件的加载不是必须的。因此对启动文件 bringup.launch 的修改示例如下:

```
$ gedit bringup.launch
$ gedit static_tf.launch
```

bringup.launch 文件修改内容示例如下:

```
<launch>
  <!-- 1. 发布静态坐标变换 -->
  <include file="$(find ebot_bringup)/launch/static_tf.launch" />
  <!-- 2. 启动底盘 -->
  <include file="$(find turn_on_wheeltec_robot)/launch/turn_on_wheeltec_
robot.launch" />
</launch>
```

static_tf.launch 文件输入内容示例如下::

```
<launch>
  <!-- 发布小车底盘与 base_footprint 的坐标变换 -->
  <node pkg="tf2_ros" type="static_transform_publisher"
name="base_footprint2base_link" args="0 0 0 0 0 base_footprint base_link" />
  <!-- 发布雷达与小车底盘的坐标变换 -->
  <node pkg="tf2_ros" type="static_transform_publisher"
name="base_link2laser_link" args="0.0 0.0 0.15 3.1415 0.0 0.0 base_link
laser_link" />
</launch>
```

修改完成后, 可删除 my_lidar.launch 中发布的雷达与小车底盘的坐标变换。

五、实验要求

1、仿真实验

加载带有激光雷达的机器人小车模型和 gazebo 仿真环境, 启动 gmapping 建图节点, 键盘控制小车在仿真环境中自由移动, 观察地图逐步生成、更新与完善的过程并检查建图效果, 待地图构建完成后, 使用 map_server 功能包将栅格地图保存为图片与配置文件。

2、实车实验

1) 修改 bringup.launch, 实现小车底盘和激光雷达传感器的联合启动。

2) 启动机器人小车底盘, 启动激光雷达传感器, 启动 hector_slam 建图节点, 键盘控制小车在教学实验场地中自由移动, 观察地图逐步生成、更新与完善的过程

并检查建图效果，待地图构建完成后，使用 `map_server` 功能包将栅格地图保存为图片与配置文件。

3) 修改 `hector_mapping` 建图配置文件中地图分辨率参数，分别设置不同分辨率进行建图，生成并保存多组不同精度的栅格地图，对比分析分辨率对地图细节的影响。

3、思考问题

1) 请分析 URDF、Rviz、Gazebo 三大工具在 ROS 仿真中的作用，并比较 Rviz 和 Gazebo 工具的区别。

2) 在建图过程中，如果 `base_footprint` 到 `laser`（雷达坐标系）之间的静态 TF 变换标定错了，对建出的地图有什么样的影响？

3) 实车实验中为什么采用 `hector_mapping` 方法建图？请给出原因，并列出 `hector_mapping` 和 `gmapping` 算法的区别。这两个算法节点分别订阅和发布了哪些坐标系之间的 TF 变换？

六、实验报告要求

1. 提交时间：两周之内提交
2. 页数：2~4 页以内
3. 报告内容：实验目的、实验步骤与结果分析、结论与问题讨论
4. 提交方式：电子版和纸质版均需提交；

电子版：实验报告和代码打包，加密压缩后，上传至网盘 <http://58.206.101.71:8266/>；

压缩包名：班级+学号+姓名，报告格式为 word 或 pdf；

纸质版：附上电子版密码，黑白双面打印即可，由各班学委收齐后交至科学馆 303。

七、参考资料

1. 张新钰，赵虚左，丘楠，郭世纯，《ROS 机器人理论与实践》，清华大学出版社
2. 课程资料：

<http://www.autolabor.com.cn/book/ROSTutorials/>

3. 课程讲解（B 站）：

https://www.bilibili.com/video/BV1Ci4y1L7ZZ/?spm_id_from=333.999.0.0&vd_source=4913ab2f45996351f46d15faddba2bfd

注：本次实验主要涉及《ROS 机器人理论与实践》第六章和第七章内容，重点参阅 6.6-6.7、7.2.1-7.2.2 内容。